

บทที่ 3

Object-Oriented System Development

3.1 บทนำ

ในอดีต การออกแบบพัฒนาระบบจะมุ่งเน้นการเขียนโปรแกรมแบบโครงสร้าง(Structural Programming) เป็นหลัก และให้ความสำคัญในเรื่องภาษาที่จะใช้เขียนโปรแกรม(Programming Language) มากกว่าที่จะคำนึงถึงรูปแบบ และความสามารถของระบบหรือโปรแกรมที่ผู้ใช้งานต้องการ การวิเคราะห์ระบบอย่างไม่ละเอียดเพียงพอทำให้เกิดความล้มเหลวในการที่เข้าถึงความต้องการที่แท้จริงของระบบ ทำให้สูญเสียเวลาและงบประมาณที่ใช้ในการพัฒนาระบบ ถึงแม้ว่าจะมีการพัฒนาความสามารถของภาษาที่ใช้ในการเขียนโปรแกรม ก็ไม่สามารถช่วยให้การออกแบบระบบง่ายขึ้นเลย อีกทั้งยังไม่สามารถช่วยให้ระบบที่พัฒนาเป็นที่ต้องการของผู้ใช้ได้

ในปัจจุบัน การออกแบบพัฒนาระบบมีความก้าวหน้าไปมาก โดยมุ่งเน้นที่การวิเคราะห์ความต้องการของผู้ใช้ อย่างละเอียดมากกว่าที่มุ่งเน้นภาษาที่ใช้ในการเขียนโปรแกรมอีกต่อไป เป็นการออกแบบระบบอย่างเป็นระบบก่อนที่จะทำการเขียนโปรแกรมจริง ๆ ทำให้โปรแกรมที่ได้พัฒนามาตรงกับความต้องการของผู้ใช้ ทำให้ผู้ใช้งานมีความพอใจในระบบที่ได้ทำการพัฒนามากขึ้น เวลาที่ใช้ในการเขียนโปรแกรมก็น้อยลงเพราะถ้าระบบไม่เป็นที่ต้องการของผู้ใช้ก็ออกแบบใหม่แทนที่จะมาการเขียนโปรแกรมใหม่ ทั้งนี้ก็ขึ้นอยู่กับวิธีการพัฒนาระบบด้วยเช่นกัน

การที่จะทำอย่างนี้ได้เราต้องสร้างโมเดลจำลอง (Abstract Model) ซึ่งเป็นการจำลองกระบวนการทำงานของระบบทั้งระบบ ซึ่งข้อดีของการมีโมเดลจำลอง มีดังนี้

1. มีความถูกต้อง ตรงกับความต้องการที่แท้จริงของผู้ใช้ (Accurate)
2. ระบบมีความสอดคล้องกันทุกส่วน (Consistent)
3. เป็นภาษากลางในการสื่อสารระหว่างผู้พัฒนาและผู้ใช้งานระบบ เพื่อใช้อธิบายการทำงานของระบบทำให้ผู้ใช้เข้าใจระบบง่ายและรวดเร็วขึ้น แทนที่จะใช้ข้อสัคคอธิบายซึ่งจะเข้าใจยากกว่า
4. สามารถเปลี่ยนแปลงแก้ไขง่าย (Change easily)
5. สนับสนุนการนำกลับมาใช้(Reusability)

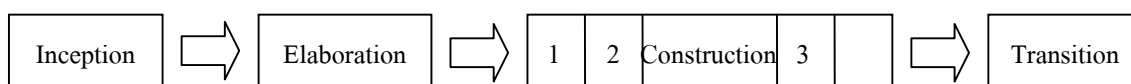
จากข้อดีเหล่านี้ จึงทำให้มีผู้พัฒนาเป็นโมเดลแบบต่าง ๆ หลายแบบ เช่น Booch, The Object Modeling Technique (OMT), OOSE/Objectory, Fusion, Coad/Yordon เป็นต้น ซึ่งเป็นความพยายามที่จะรวมการพัฒนาออกแบบระบบกับการเขียนโปรแกรมไว้ด้วยกัน มีผลทำให้เกิดความสอดคล้องระหว่างกัน ซึ่งแต่ละโมเดลจะมีสัญลักษณ์ที่ใช้อธิบายระบบ(Notation), เครื่องมือที่ใช้อธิบายระบบ(Tool) และ

กระบวนการที่ใช้พัฒนาโปรแกรม (Process) แตกต่างกัน ซึ่งโมเดลที่นิยมใช้กันมากได้แก่ Unified Modeling Language (UML) ซึ่งเป็นการรวมเอาข้อดีของแต่ละโมเดลข้างต้นมาไว้ด้วยกัน

Unified Modeling Language (UML) โดยตัวภาษาแล้วจะเป็นภาษาสัญลักษณ์ที่ใช้อธิบายการทำงานของระบบ เป็นโมเดลเชิงวัตถุ (Object-Oriented) ซึ่งผู้พัฒนาคือ Grady Booch, James Rumbaugh และ Ivar Jacobson มีกระบวนการที่ครอบคลุมถึงการวิเคราะห์ (Analysis) ระบบจนถึงการออกแบบ (Design) จนเป็นที่ยอมรับ โดยได้มีการรวมกลุ่มบริษัทเป็นกลุ่มที่มีชื่อว่า “Object Management Group” ประกอบด้วยบริษัทดังต่อไปนี้ Digital Equipment Corporation, Hewlet Packard, I-Logix, Intellicorp, IBM, Texas Instrument, ICON Computing, Microsoft, Oracle, MCI System Software และ Unisys ยอมรับมาตรฐานของ UML มาใช้ และได้มีการยอมรับ UML เป็นมาตรฐาน Defacto ด้วย

3.2 กระบวนการพัฒนาระบบตามแบบวิธี Rational Unified Process หรือ Rational Objectory Process

การพัฒนาระบบตามแบบวิธี Rational Unified Process เป็นกระบวนการที่ครอบคลุมกระบวนการพัฒนาระบบทั้งหมด โดยมองเป็นกระบวนการในระดับมหัพภาคโดยการพิจารณาทั้งงานด้านการบริหารและงานด้านเทคนิค กระบวนการพัฒนามีลักษณะการทำซ้ำ (Iterative) และการเพิ่มขึ้น (Incremental) ดังนั้นงานที่ทำจะไม่มีมากในคราวเดียวในตอนสุดท้ายของโปรเจกต์ แต่จะมีการแบ่งงานออกเป็นช่วง ๆ (Phase) ในช่วงของการสร้างระบบ (Construction Phase) การทดสอบและการรวมระบบย่อยเข้ากับระบบรวม จะมีการทำซ้ำหลาย ๆ ครั้ง เพื่อให้ได้โปรแกรมที่มีคุณภาพและตรงตามความต้องการ ในการทำซ้ำแต่ละรอบ จะประกอบด้วย การวิเคราะห์ (Analysis), การออกแบบ (Design), การอิมพลีเมนต์ (Implement) และการทดสอบ (Testing) ช่วงของการพัฒนาระบบจะแสดงดังรูป



รูปที่ 3-1 แสดงการพัฒนาตามแบบวิธี Rational Unified Process

ช่วงของการพัฒนาจะประกอบด้วย

- อินเซพชั่น (Inception) กำหนดขอบเขตและจุดมุ่งหมายของโครงการ
- อีลาบอเรชั่น (Elaboration) รวบรวมข้อมูลเพื่อหาความเป็นไปได้ของโปรเจกต์และความต้องการทรัพยากรต่าง ๆ การกำหนดฟังก์ชันและโครงสร้างพื้นฐาน รวมทั้งวางแผนเพื่อการพัฒนาช่วงต่อไป
- คอนสตรัคชั่น (Construction) การสร้างระบบ โดยมีการทำซ้ำทั้ง วิเคราะห์, ออกแบบ, การเขียนโค้ดและการทดสอบ

- ทรานซิชัน (Transition) ทำการส่งระบบไปยังผู้ใช้ อาจมีการทดสอบเบต้า (Beta testing), การทดสอบประสิทธิภาพ (Performance Tuning) และการสอนการใช้โปรแกรมแก่ผู้ใช้ (User training)

อย่างไรก็ตาม เราสามารถมีการทำซ้ำได้ในทุกช่วงการพัฒนาขึ้นกับว่าระบบใหญ่แค่ไหน หรือต้องการรายละเอียดมากแค่ไหน แต่ที่เน้นช่วงการสร้างระบบ เพราะเป็นช่วงที่มีความสำคัญต่อระบบ การพัฒนาแต่ละช่วงมีรายละเอียดดังนี้

3.2.1 อินเซพชัน เฟส (Inception Phase)

เป็นการเก็บข้อมูลพื้นฐานเกี่ยวกับระบบที่ต้องการ โดยจะมีความเกี่ยวข้องกับฟังก์ชันการทำงานต่าง ๆ ความสารถ ประสิทธิภาพ เทคโนโลยีที่ใช้ และคุณสมบัติอื่น ๆ อีกทั้งยังเป็นการกำหนดแนวคิดเพิ่มเติมและแสดงวิธีที่ใช้ในการพัฒนาในขั้นตอนต่อไป และแสดงวิธีการที่ทำให้ระบบมีความสามารถมากขึ้น

ผลลัพธ์ได้จากกระบวนการนี้จะปรากฏอยู่ในรูปของแผนงานโดยรวม ซึ่งแสดงว่าจะต้องสร้างอะไรขึ้นมาบ้าง กำหนดว่าจะสร้างได้อย่างไรและมีการทำงานได้อย่างไร กระบวนการนี้จำเป็นต้องมีทักษะในการวิเคราะห์ระบบให้ออกมาในรูปของฟังก์ชันหลักของระบบและ Actor ซึ่งอธิบายในรูปของ Use case view และยังต้องมีการวางแผนด้านงบประมาณ ค่าใช้จ่ายในการพัฒนาระบบ ความสามารถทางการตลาด การวิเคราะห์ความเสี่ยง และผลิตภัณฑ์ของคู่แข่ง ในกรณีการพัฒนาระบบเพื่อธุรกิจ

3.2.2 อีลาบอเรชัน เฟส (Elaboration Phase)

ประกอบด้วยรายละเอียดของการวิเคราะห์ระบบ การกำหนดและวางแผนก่อนการทำงานในขั้นตอนต่าง ๆ ทั้งในส่วนของการทำงานและขอบเขตของปัญหา โดยจะออกมาในรูปของไดอะแกรมต่าง ๆ เช่น Use case diagram, Class diagram, Dynamic diagram และ Deployment diagram

3.2.3 คอนสตรัคชัน เฟส (Construction Phase)

เป็นการพัฒนาระบบจริงโดยการเขียนโปรแกรม ซึ่งมีการพัฒนาแบบซ้ำ ๆ และเพิ่มขึ้นเรื่อย ๆ กระบวนการต่าง ๆ ที่ทำซ้ำ ประกอบด้วย ขั้นตอนการ วิเคราะห์, ออกแบบ, เขียนโปรแกรมและการทดสอบ และทำการเพิ่มรวมเป็นระบบใหญ่ขึ้น จนได้ระบบที่ต้องการผลลัพธ์ของการทำงานในช่วงนี้คือระบบที่ต้องการ

3.2.4 ทรานซิชัน เฟส (Transition Phase)

เป็นกระบวนการส่งผลิตภัณฑ์ไปสู่ผู้ใช้งานจริงรวมไปถึงการหาตลาด การเปิดตัว และบำรุงรักษา และการสอนการใช้โปรแกรม และจัดทำคู่มือการใช้โปรแกรม

3.3 ส่วนประกอบของ UML

UML ประกอบด้วยส่วนต่าง ๆ ดังนี้

- วิว (View) จะแสดงมุมมองหรือแง่มุมของระบบที่เราจะออกแบบ โดยใช้ไดอะแกรมต่าง ๆ ในการอธิบาย ซึ่ง View เป็นส่วนที่ใช้แสดงไดอะแกรม
- ไดอะแกรม (Diagram) จะแสดงความสัมพันธ์ของระบบในแต่ละ view เป็นการใช้กราฟในการอธิบาย แต่ละไดอะแกรม จะใช้กราฟต่าง ๆ กัน และขึ้นอยู่กับ view ด้วย
- โมเดลอีลิเมนต์ (Model element) เป็นสัญลักษณ์ที่ใช้ในแต่ละไดอะแกรม เพื่อแสดงหรือเป็นตัวแทนของสิ่งต่าง ๆ เช่น Class , Object, Message และ ความสัมพันธ์ (relationship) เป็นต้น
- เจเนอรัล เมคานิซึม (General Mechanism) เป็นส่วนแสดงคอมเมนต์เพิ่มเติม(Extra comment), ข้อมูลอื่น ๆ ที่จำเป็น หรือความหมายของ model element

3.3.1 วิว (View)

ระบบงานทั้งหมดอาจมีหลายส่วนที่ต้องพิจารณา เพราะอาจมีขอบข่ายงานที่กว้างและซับซ้อน การอธิบายกระบวนการทำงานต่าง ๆ ของระบบไม่สามารถอธิบายได้เพียงแค่มุมมองเดียว (View) ดังนั้น การมองระบบควรจะต้องมีมุมมองต่าง ๆ กัน เช่น มุมมองด้าน ฟังก์ชันนอล (Functional), นอนฟังก์ชันนอล (Nonfunctional), มุมมองขององค์กร เป็นต้น ซึ่งแต่ละ Diagram สามารถที่จะมีมุมมองได้มากกว่าหนึ่งมุมมอง ดังนั้นการมีมุมมองหลายมุมมองก็เพื่อมาอธิบายภาพรวมของระบบ โดยอาจเป็นมุมมองของผู้ใช้ระบบ ผู้เขียนโปรแกรมพัฒนาระบบ ซึ่งแต่ละมุมมองทำให้ผู้ทำความเข้าใจระบบ เข้าใจระบบในแง่มุมที่ต่าง ๆ กัน

มุมมอง (View) ต่าง ๆ ของ UML มีดังนี้

3.3.1.1 ยูสเคสวิว (Use-case view)

เป็นการมองระบบจากผู้ใช้ภายนอก หรือผู้ใช้ระบบ ซึ่งไดอะแกรมที่ใช้อธิบาย คือ ยูสเคสไดอะแกรม (Use-case diagram) หรือบางครั้งโดย activity diagram ตัวอย่างผู้ใช้งาน เช่น ลูกค้า, ผู้ออกแบบ, ผู้ทดสอบระบบ, นักเรียน, อาจารย์ เป็นต้น use case ใน Use-case diagram เป็นการทำงานของระบบที่ผู้ใช้ต้องการ ซึ่งได้มาจากการสำรวจความต้องการของผู้ใช้ Use-case diagram เป็นตัวกำหนดเป้าหมายของระบบ จึงเป็นส่วนกลางของ view อื่น ๆ ที่จะต้องมีการทำงานต่าง ๆ ครอบคลุมที่กำหนดไว้ใน Use-case diagram

3.3.1.2 ลอจิกัลวิว (Logical view)

ใช้อธิบายว่าสามารถที่จะจัดการทำงานของระบบให้เป็นไปตามที่ต้องการได้อย่างไรและมีบริการอะไรให้กับผู้ใช้บ้าง Logical view ต่างจาก use case view เนื่องจากเป็นมุมมองของผู้ออกแบบและพัฒนา ระบบ โดยจะแสดงในรูปแบบของโครงสร้างแบบสถิต (Static) เช่น คลาส (Class), ออบเจกต์ (Object), ความสัมพันธ์ระหว่างการทำงานร่วมกันแบบไดนามิก(dynamic collaboration) ซึ่งเกิดเมื่อออบเจกต์ ส่งเมสเสจระหว่างการทำงาน

โครงสร้างแบบสถิตจะอธิบายโดยใช้ คลาสไดอะแกรม(Class diagram) และ ออบเจกต์ไดอะแกรม (Object diagram) ส่วนการทำงานร่วมกันแบบไดนามิกจะอธิบายโดยใช้ สเตตไดอะ-แกรม (state diagram),ซีควเन्ซ์ไดอะแกรม (sequence diagram), คีอลเลโบเรชันไดอะแกรม (collaboration diagram) และแอคทิวิตีไดอะแกรม (activity diagram)

3.3.1.3 คอมโพเนนท์วิว (Component view)

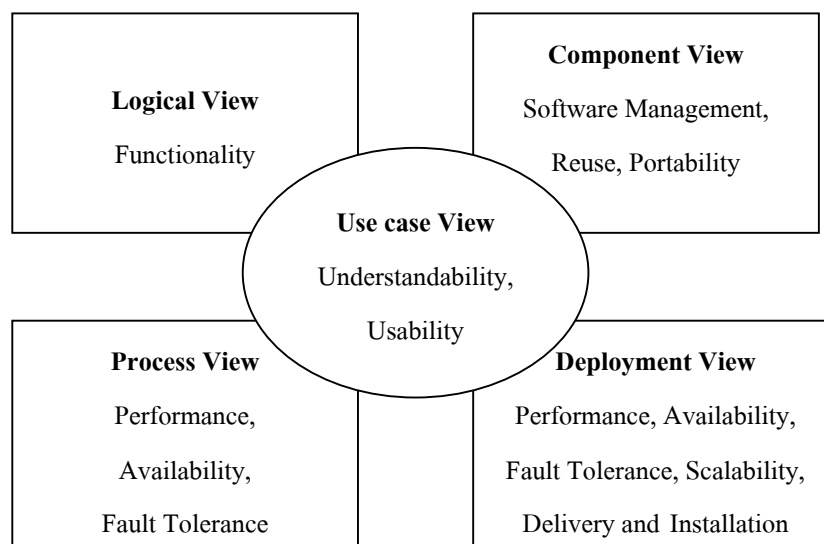
บอกถึงการสร้างและความขึ้นต่อกันของโมดูล (Module) ซึ่งเป็นส่วนที่ผู้พัฒนาระบบต้องคำนึงถึง ว่าในแต่ละคอมโพเนนต์ประกอบด้วยโครงสร้างและความขึ้นต่อกันตลอดจนข้อมูลต่าง ๆ เช่นความต้องการทรัพยากรของคอมโพเนนต์นั้น มีอะไรบ้าง โดยใช้คอมโพเนนต์ไดอะแกรม (Component diagram) ในการอธิบาย

3.3.1.4 ดีพลอยเมนต์ (Deployment view)

เป็นการแสดงการจัดระบบในระดับกายภาพ (Physical) ให้เหมาะสม เช่นการเชื่อมต่อระหว่างคอมพิวเตอร์และโหนดต่าง ๆ และรวมถึงการแมป(map) คอมโพเนนต์ต่าง ๆ ในระดับ โครงสร้างทางกายภาพ เช่น ลำดับการ ของ หรือโปรแกรมในแต่ละเครื่องคอมพิวเตอร์ ใช้สำหรับผู้พัฒนาระบบ, ผู้ร่วมพัฒนาระบบ, ผู้ทดสอบระบบ อธิบายโดย Deployment diagram

3.3.1.5 โพรเซส วิว (Process view)

จะแสดงการทำงานร่วมกันและการติดต่อกันของส่วนต่าง ๆ ในระบบ



รูปที่ 3-2 แสดงสถาปัตยกรรมของวิว

3.3.2 ไดอะแกรม (Diagram)

เป็นกราฟซึ่งแสดงโดยสัญลักษณ์ที่จัดเรียงเพื่อใช้อธิบายระบบในมุมมองต่าง ๆ ในระบบหนึ่ง ๆ จะประกอบไปด้วย หลาย ๆ ไดอะแกรม แต่ละไดอะแกรมยังสามารถมองในหลาย ๆ มุมมองด้วย

ใน UML จะมีไดอะแกรมต่าง ๆ ดังนี้

3.3.2.1 ยูสเคส ไดอะแกรม (Use-case diagram)

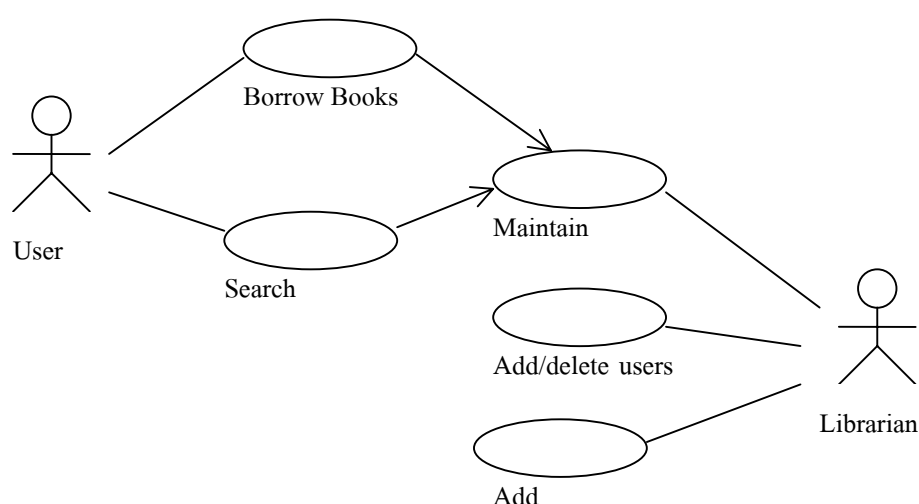
ยูสเคสไดอะแกรมจะแสดงความสัมพันธ์ระหว่างผู้ใช้งาน กับระบบ ซึ่งจะมีผู้กระทำจากภายนอก (Actor) กับระบบ โดยติดต่อผ่าน use case ต่าง ๆ ที่เกี่ยวข้อง และจะใช้ในการสื่อสารกับผู้ใช้งาน

เพื่ออธิบายถึงฟังก์ชันการทำงานของระบบ ซึ่งยูสเคส ใน Use case diagram ซึ่งก็คือการทำงานต่าง ๆ ที่ผู้ใช้งานต้องการ ซึ่งจะได้มาจากการสอบถามจากผู้ใช้งาน

แอ็กเตอร์ ไม่ใช่ส่วนประกอบของระบบ แต่เป็นส่วนที่ใช้ติดต่อกับระบบ ซึ่งอาจเป็นเพียงการป้อนข้อมูลเข้าสู่ระบบ หรือการส่งข้อมูลออกจากระบบ หรืออาจเป็นทั้งสองอย่าง การที่จะหาแอ็กเตอร์ของระบบ อาจต้องตอบคำถามต่อไปนี้ได้ เช่น

1. ใครเป็นผู้ใช้งานระบบ
2. ใครมีความเหมาะสมที่จะใช้งานระบบ
3. ระบบมีการใช้งานจากภายนอกหรือไม่

ซึ่งคำตอบก็คือแอ็กเตอร์

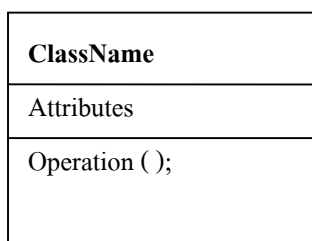


รูปที่ 3-3 แสดงแผนภาพ ยูสเคส ระบบห้องสมุด

3.3.2.2 คลาสไดอะแกรม (Class diagram)

แสดงโครงสร้างของส่วนที่ไม่เปลี่ยนแปลงของระบบ ในมุมมองของผู้พัฒนาระบบ ซึ่งสามารถแสดงความสัมพันธ์ได้หลายวิธี ได้แก่ Association (เชื่อมต่อระหว่างกัน), dependent (การพึ่งพาเรียกใช้คลาสอื่นๆ), specialized (ความเป็นลักษณะเฉพาะของคลาสอื่นๆ), package (รวมเป็นหน่วย) ความสัมพันธ์ระหว่างคลาสดังกล่าวเหล่านี้ จะถูกแสดงโดยคลาสไดอะแกรม โดยรวมเข้าเป็นโครงสร้างภายในของคลาสเป็นกลุ่มแอตทริบิวต์(attribute) และ กลุ่มโอเปอเรชัน (operation) ในระบบหนึ่งสามารถประกอบด้วยหลายคลาสไดอะแกรม

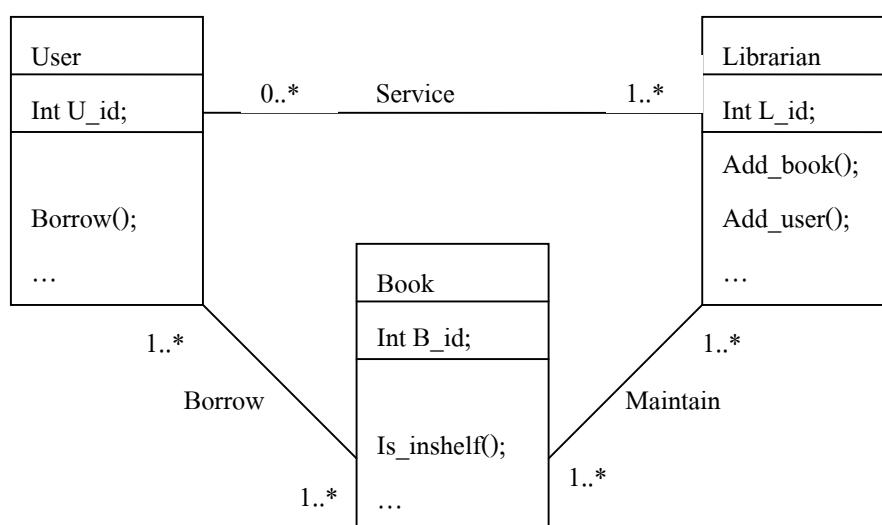
คลาส คือ กลุ่มของออบเจกต์ที่มีคุณสมบัติ(Attributes)และพฤติกรรม(behavior)ร่วมกัน



รูปที่ 3-4 แสดงสัญลักษณ์คลาส

ความสัมพันธ์ต่าง ๆ ของคลาสไดอะแกรม

- 1 หมายถึงจะมีออบเจกต์ในคลาสไดอะแกรมได้หนึ่งออบเจกต์เท่านั้น
- 0..1 หมายถึงจะมีออบเจกต์ในคลาสไดอะแกรมได้แค่หนึ่งหรืออาจจะไม่มีก็ได้
- M..N หมายถึงจะมีออบเจกต์ในคลาสไดอะแกรมได้ตั้งแต่ M ถึง N (เมื่อ M, N เป็นจำนวนเต็มบวก)
- * หมายถึงจะมีออบเจกต์ในคลาสไดอะแกรมได้ตั้งแต่ศูนย์ขึ้นไป
- 0..* หมายถึงจะมีออบเจกต์ในคลาสไดอะแกรมได้ตั้งแต่ศูนย์ขึ้นไป
- 1..* หมายถึงจะมีออบเจกต์ในคลาสไดอะแกรมได้ตั้งแต่หนึ่งขึ้นไป



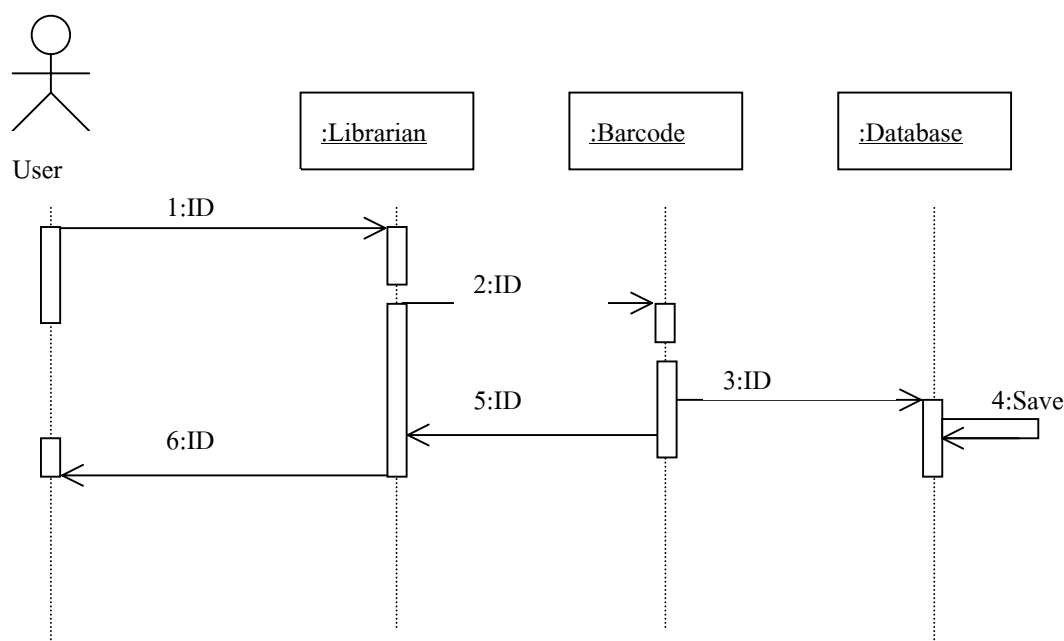
รูปที่ 3-5 แสดงตัวอย่าง Class diagram ของระบบห้องสมุด

3.3.2.3 ออบเจ็กต์ไดอะแกรม (Object diagram)

Object diagram ได้มาจาก class diagram และมีสัญลักษณ์ส่วนใหญ่เหมือนกัน ความแตกต่างระหว่าง 2 ไดอะแกรมนี้ คือ ออบเจ็กต์ไดอะแกรมจะแสดงออบเจ็กต์ซึ่งเป็นตัวแทนของคลาสแทนที่จะเป็นคลาสจริง ๆ ดังนั้น ออบเจ็กต์ไดอะแกรมจะแสดงความเป็นไปได้ของระบบขณะกำลังถูกใช้งาน มีประโยชน์คือใช้เป็นตัวอย่างสำหรับคลาสไดอะแกรมที่ซับซ้อน

3.3.2.4 ซีควเอนซ์ไดอะแกรม (Sequence diagram)

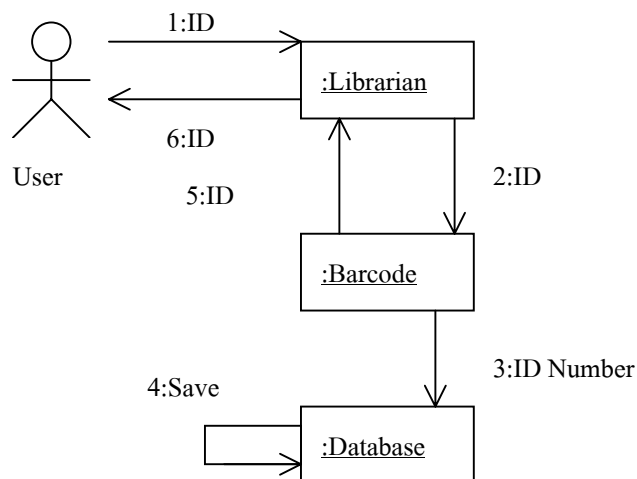
แสดงการทำงานที่เป็นไคนามิกของออบเจ็กต์ต่าง ๆ หรือแสดงเมสเสจระหว่างออบเจ็กต์ ที่เน้นไปที่ลำดับก่อนหลังของการทำงานโดยใช้สัญลักษณ์ที่เรียกว่า ออบเจ็กต์ ซีควเอนซ์ ชาร์ต (Object sequence chart)



รูปที่ 3-6 แสดง Sequence Diagram ของการเพิ่มผู้ใช้บริการห้องสมุด

3.3.2.5 คอลลาโบเรชันไดอะแกรม (Collaboration diagram)

อธิบายการทำงานร่วมกันของออบเจ็กต์ที่มีการส่งเมสเสจคล้าย ๆ กับ sequence diagram แต่เน้นที่ตัวออบเจ็กต์มากกว่าลำดับก่อนหลัง และเน้นการส่งเมสเสจระหว่างออบเจ็กต์



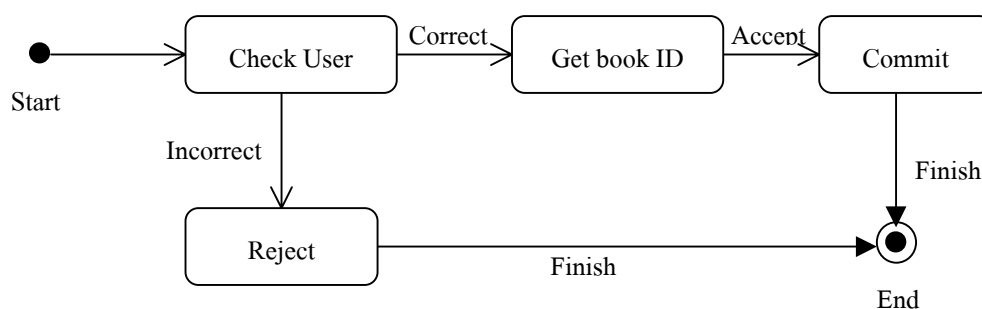
รูปที่ 3-7 แสดง Collaboration diagram ของการเพิ่มผู้ใช้ห้องสมุด

3.3.2.6 สเตตไดอะแกรม (State diagram)

อธิบายประกอบคลาสไดอะแกรม โดยจะแสดงทุก ๆ สถานะที่เป็นไปได้และเหตุการณ์ที่ทำให้ ออบเจกต์ต่าง ๆ เกิดการเปลี่ยนแปลง โดยเหตุการณ์ที่ทำให้ ออบเจกต์เกิดความเปลี่ยนแปลงมาจาก ออบเจกต์อื่นส่งมาเสมอเสมอ การเปลี่ยนสถานะเรียกว่าทรานซิชัน (Transition) โดยมีแอ็กชัน(action) ทำให้เกิดการเปลี่ยนสถานะ state diagram ไม่จำเป็นต้องไม่จำเป็นต้องวาดทุกคลาสแต่จะใช้สำหรับคลาสที่มีการกำหนดสถานะไว้ชัดเจนและเมื่อมีการเปลี่ยนแปลงเท่านั้น

3.3.2.7 แอกติวิตีไดอะแกรม (Activity diagram)

แสดงลำดับการไหลของกิจกรรมต่าง ๆ โดยจะอธิบายกิจกรรมต่าง ๆ ในลักษณะของการกระทำ จะมีเงื่อนไขและการตัดสินใจกำหนดไว้เพื่อควบคุมการไหลของกิจกรรมรวมถึงเมสเสจที่รับส่งระหว่างแต่ละกิจกรรม



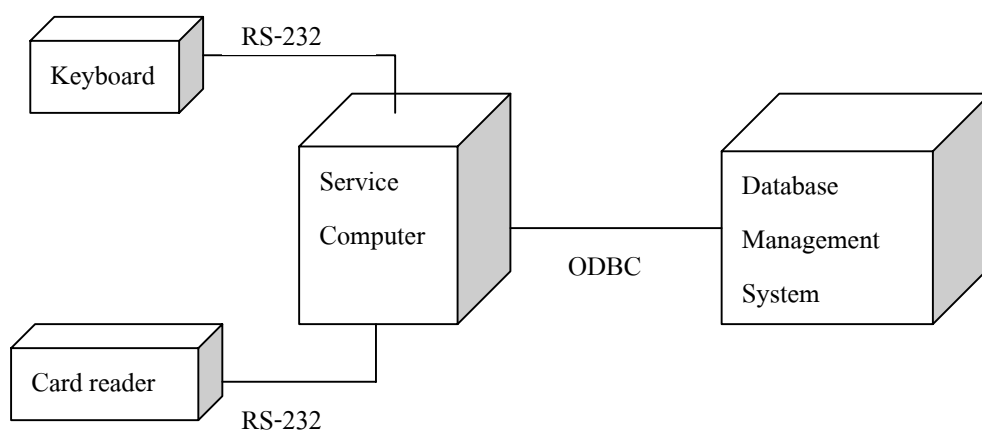
รูปที่ 3-8 แสดง Activity diagram ของการยืมหนังสือ

3.3.2.8 คอมโพเนนท์ (Component diagram)

แสดงโครงสร้างทางกายภาพของโค้ด อาจเป็นส่วนประกอบซอสโค้ด คอมโพเนนต์ประกอบด้วยข้อมูลเกี่ยวกับ Logical class ของ component นั้น ในไดอะแกรมแสดงความสัมพันธ์หรือความพึ่งพากันของคอมโพเนนท์ จะช่วยในการวิเคราะห์ว่าเมื่อเกิดการเปลี่ยนแปลงของคอมโพเนนต์หนึ่งจะมีผลต่ออีกคอมโพเนนต์อื่นๆ อย่างไรในโปรแกรม

3.3.2.9 ดีพลอยเมนต์ไดอะแกรม (Deployment diagram)

แสดงสถาปัตยกรรมทางกายภาพของฮาร์ดแวร์และซอฟต์แวร์ที่ใช้ในระบบ ซึ่งสามารถแสดงเป็นคอมพิวเตอร์และโหนดที่เชื่อมต่อถึงกัน ภายในโหนดยังสามารถมีคอมโพเนนต์หรือออบเจกต์ที่สามารถปฏิบัติกับโหนดเพื่อแสดงว่าโปรแกรมส่วนใดปฏิบัติบนโหนดใด และความสัมพันธ์ระหว่างคอมโพเนนต์ที่อยู่บนโหนด



รูปที่ 3-9 แสดง *Deployment diagram* ของระบบห้องสมุด